

FULL STACK DEVELOPMENT

FASE 5 | AULA 03 -

A CAMADA ENTIDADES NA CLEAN ARCHITECTURE

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	10
REFERÊNCIAS.....	11

EMSE

O QUE VEM POR AÍ?

A Clean Architecture dá ênfase à construção de uma estrutura que mantém independência entre as responsabilidades da Regra de Negócio e as responsabilidades de interação com o mundo exterior. As camadas internas, de Casos de Uso e Entidades, são baseadas em “o que” o software faz.

Vamos falar sobre a camada de entidades e como ela funciona para isolar os elementos utilizados na implementação das Regras de Negócio.



HANDS ON

Agora, veremos o que é a camada de entidades, para que servem e como fazer a criação de entidades usando alguns exemplos e analisando algumas regras da Arquitetura Limpa.

Para essa aula, temos um link para você poder treinar o conteúdo ensinado. Acesse: [Base de dados do GitHub](#).



SAIBA MAIS

Se até agora falamos sobre a arquitetura limpa de forma mais ampla, com todos os componentes vistos de forma integrada, agora vamos detalhar as camadas de software, uma a uma. Vamos começar com a mais interna, a de entidades; e começaremos por ela porque é a camada que define os elementos que a Regra de Negócio usa.

É importante levantarmos, neste momento, um ponto relativo à construção do software: no momento do desenho da solução, não vamos definir **primeiro** a camada de entidades e **depois** as outras camadas. A criação do software tem que ser feita de forma integrada, se possível guiada por testes, e isso faz com que o pensamento seja sempre voltado às camadas e responsabilidades, e **onde** cada parte da implementação deve ser colocada.

No momento da análise do problema, levantamos os requisitos que, usualmente, se tornam Casos de Uso, documentando o que o software precisa fazer em cada momento que houver uma interação de um agente externo com o software. Esse agente externo pode ser uma pessoa, um outro software ou algum tipo de interação que ainda não conhecemos. Mas, para o Caso de Uso ser detalhado, conhecer este agente externo não é o mais importante, e sim o que ele precisa fazer, o que precisa de resposta, e qual o resultado dessa ação do sistema.

Para o funcionamento de um Caso de Uso, construímos uma lógica que trabalha com diversos elementos de dados e/ou responsabilidades que tem o objetivo de representar um ator no processo. Ficou difícil? Vamos exemplificar:

Um caso de uso de uma compra em um mercado pode ser descrito como: “Uma **pessoa** entra em um **mercado**, pega um **produto** e leva para o **caixa**, onde é gerada uma **compra** com um valor total, e então a **pessoa** faz o **pagamento** e leva o produto para casa”.

Identificamos aqui alguns desses elementos ou **entidades**: a **pessoa**, o **mercado**, o **produto**, o **caixa** e o **pagamento**. Cada um deles tem um conjunto de dados que os especificam, e podem ter regras de validação interna para definir se os dados estão corretos e consistentes, bem como algumas outras regras que definem o relacionamento com outros elementos.

Um **pagamento**, por exemplo, não pode ter o valor negativo; o **produto** precisa ter um preço. Um **pagamento** não pode ser feito se na **compra** não houver ao menos um produto. Todas essas são regras relacionadas aos elementos e aos relacionamentos entre si, mas não relacionados a um processo de negócio complexo, com mais de um passo, como “Efetuar uma Venda”.

Cada uma dessas entidades têm um significado único, e podem ser especializações de uma outra entidade, podem ser compostas de outras entidades, ou puramente definidas em si. Essas entidades são definidas por seu papel na Regra de Negócio, e nomeadas de acordo com o que representam. Uma das principais estratégias usadas na nomeação das entidades é se basear na linguagem ubíqua, de acordo com o Domain Driven Design, e criar as entidades com o mesmo nome usado pelas pessoas que conhecem o domínio. Usar nomes incomuns vai facilitar a implementação de entidades desconexas da realidade, e consequentemente, vai criar uma camada de entidades desconectada da realidade.

Essas entidades são implementadas de acordo com a linguagem de programação selecionada, e usualmente são classes, tipos ou structs. São definições de **tipos de dados**, porque cada um desses elementos deve ser diferenciado dos outros por suas características, ações e estados internos.

Precisamos tomar dois cuidados aqui: o primeiro é não entender as entidades como “a camada de classes de dados”, e o outro é não basear a camada de entidades na modelagem do Repositório de Dados, refletindo a ideia comum, de muitos frameworks ou soluções de software, de conexão direta entre as entidades do Repositório de Dados e as classes de entidades, em um modelo “um-para-um”.

No primeiro caso, vamos incorrer no erro de criar classes que apenas servem como estruturas de armazenamento e validação de dados, totalmente inertes em relação ao papel que têm dentro da estrutura. E, ao contrário, precisamos pensar em cada entidade em relação ao papel na estrutura. Porque a definição das entidades é importante para fazer os casos de uso funcionar, e para representar os atores envolvidos nos processos de negócio. Se pensamos apenas na ideia de que “entidades são as abstrações de dados”, não conseguimos cumprir um dos principais objetivos da Arquitetura Limpa, que é a identificação de responsabilidades em cada uma das camadas.

No segundo caso, o erro é estrutural: não conseguimos definir uma camada de entidades **desacoplada** do Repositório de Dados se as entidades mapeiam diretamente as estruturas deste. A restrição, neste caso, se impõe de formas diretas e indiretas: a entidade vai espelhar os tipos de dados da implementação externa, vai refletir nomes que não podem ser significantes para a estrutura do software, e podemos terceirizar alguns tipos de validações para o Repositório de Dados, onde ele não é o responsável por este trabalho. Fica impossível implementar a Arquitetura Limpa em uma estrutura onde este acoplamento é imperativo, ou onde a obrigação do software é apenas servir como uma interface para um Repositório de Dados.

Como falamos muito nas aulas anteriores, a ideia é que as camadas interiores funcionem de forma **independente** e **desconectada** do resto da aplicação. O objetivo da Arquitetura Limpa é estruturar o software de forma a ser simples de entender, de alterar e de estender. E o melhor jeito de fazer isso é mantendo as regras e implementações explícitas e organizadas de acordo com sua função, em sua camada isolada.

Vamos ver algumas implementações de entidades, baseadas no exemplo que usamos acima. Algumas decisões que vamos tomar podem não parecer fazer sentido no mundo real, mas fazem sentido no **contexto** da nossa aplicação.

```
1 class CPF {
2     private readonly value: string;
3
4     constructor(numero: string) {
5         if (!this.validate(numero)) {
6             throw new Error("Valor inválido");
7         }
8         this.value = numero;
9     }
10
11     private validate(value: string): boolean {
12         // TODO: fazer a validação
13         return true;
14     }
15
16     get numero(): string {
17         return this.value;
18     }
19 }
20
21
22 type Pessoa = {
23     cpf: CPF;
24 }
```

Figura 1 – Código – Definição da Pessoa

Fonte: Elaborado pelo autor (2023)

Aqui temos a definição da **Pessoa**, e para os nossos Casos de Uso, só precisamos do número de CPF, que é necessário para a geração da Nota Fiscal. Mas o número de CPF precisa ser válido, e por isso temos outra entidade: **CPF**. Implementamos nessa entidade a validação do valor informado. Assim, se no momento de criar uma Pessoa o valor do CPF for inválido, a criação do objeto Pessoa retorna erro.

Algumas lições podem ser obtidas desse exemplo: a primeira é de que implementamos na entidade apenas o que é necessário; na realidade, uma pessoa é caracterizada por informações como nome, idade, endereço, mas estas informações são inúteis em nosso contexto, e por isso não as consideramos. Outra lição é de que implementamos junto à definição da entidade as suas regras de validação de acordo com o que define uma entidade como consistente. No exemplo acima, o tipo Pessoa tem um campo do tipo CPF; ao criar o CPF, é importante que o valor esteja válido, e por isso uma das saídas encontradas é isolar o CPF como uma classe, com a validação interna, e ao criar um CPF automaticamente garantimos que o valor seja validado, e caso não o seja, um erro é retornado.

Entidades e os Casos de Uso

Se as entidades definem a estrutura dos elementos da regra de negócio de um software, elas não são definidas por si, e sim dentro do contexto de casos de uso; não vamos implementar uma entidade porque acreditamos que seja necessária, e sim implementamos a entidade de acordo com o que é necessário. Ao mesmo tempo, uma entidade que é identificada nos casos de uso e implementada corretamente serve como uma abstração de um ator para este caso de uso.

As entidades, criadas corretamente, são testáveis em dois contextos: da validade dos dados internos de acordo com sua implementação, e na consistência de dados dentro de uma regra de negócio. Um exemplo nos ajudará a entender isso melhor.

Um Estudante pode se matricular em uma Disciplina e isso pode ser validado dentro das regras das entidades de Estudante, que tem seus dados próprios (nome, idade, endereço) e das regras da entidade de Disciplina (nome, oferta atual). Neste momento, temos dados que fazem com que os objetos dessas entidades possam ser

criados corretamente e testados. É possível criar um objeto do tipo Estudante usando dados válidos e garantir que este funciona corretamente.

Mas na regra de negócio definimos que, para um Estudante se matricular numa Disciplina, ele precisa ter cursado disciplinas anteriores (pré-requisitos), e não pode ter cursado esta Disciplina e ter a terminado com aprovação. Essas informações não estão definidas nas entidades, porque não estão relacionadas diretamente ao estado delas, e sim a uma regra complexa definida como um Caso de Uso. Isoladamente, considerando apenas os dados e a estrutura de um elemento da camada de Entidade, um Estudante pode ter terminado uma Disciplina. Para o Caso de Uso de matrícula em uma disciplina, isso pode causar um erro.

Então, percebemos que o Caso de Uso nos orienta a implementar alterações nas entidades, e uma delas é implementar uma lista de Disciplinas já cursadas por este Estudante, para que seja possível validar se o Estudante pode se matricular corretamente.

Este tipo de alteração existe porque, como falamos anteriormente, **a construção das entidades deve seguir a necessidade das regras de negócio**, e a representante dessas regras de negócio, neste momento, são os elementos da camada de Casos de Uso, sobre a qual falaremos futuramente.

O QUE VOCÊ VIU NESTA AULA?

A primeira camada da Arquitetura Limpa que analisamos é a de entidades, que é a camada central do Diagrama. Nessa camada nós implementamos os elementos do software que representam os dados que são manipulados pela regra de negócio e suas responsabilidades: uma pessoa, uma disciplina, uma venda, um produto.

Essas abstrações são implementadas para definir o que os casos de uso devem manipular, e quais as regras internas de cada um desses elementos, como definimos a validade do estado interno de cada um desses elementos, como estes elementos se comunicam entre si e quais os limites de responsabilidade.

Esses dados não se relacionam diretamente com as entidades de uma camada de Repositório de Dados ou outra camada não relacionada aos Casos de Uso, e não estão conectadas a essa camada. Devem ser implementadas de acordo com o identificado no momento de entendimento da Regra de Negócio, e devem conter apenas o necessário para fazer os Casos de uso funcionarem corretamente.

REFERÊNCIAS

EVANS, E. **Domain-Driven Design: Tackling Complexity in the Heart of Software**. [s.l.]: Addison-Wesley Professional, 2003.

MARTIN, R. **Arquitetura Limpa**. Rio de Janeiro: Alta Books Editora, 2019.

MARTIN, R. **Clean Code: A Handbook of Agile Software Craftsmanship**. [s.l.]: Pearson, 2008.

EVANS

PALAVRAS-CHAVE

Entidades. Camadas Internas. Arquitetura Limpa.

ENTRADA

POSTECH